Automated First-Break Picking

from Hardrock Seismic Reflection Data

Rufat Mahmudov
Junior ML/DL Algorithm Developer — Technical Task
eiGroup LLC · April 2026

Problem: First-break picking

The first arrival of a seismic wave at each receiver

What is it?

For each recorded trace, identify the exact time sample where the seismic wave first arrives.

Why does it matter?

Inputs for near-surface velocity models, refraction statics, and tomographic inversion.

Why is it hard?

Noisy far-offset traces, variable source-receiver geometry, and different picking conventions across surveys.

Scale

A single survey has millions of traces; manual picking is days to weeks of an expert's time.

The red curve is the first-break pick — one value per trace, aligned with wavefront onset.

The task

Automated, robust, generalizable

> *Develop an algorithm to automatically identify seismic first breaks* ***across different seismic assets, regardless of varying SNR and variation of first-break line behavior.***
>
> — eiGroup LLC technical task description

DATA

Four 3D hardrock seismic surveys: Brunswick, Halfmile, Lalor, Sudbury

METHOD

Design & implement an approach of your choice. Document methodology.

DELIVER

Clean code + short presentation (25–35 min) covering results.

Four hardrock seismic surveys

Published benchmark from the Canadian Shield mining camps

Survey	Location	Sampling	Sensors	Pick convention
Lalor	Manitoba, Canada	1 ms	MEMS accelerometers	Onset
Brunswick	New Brunswick, Canada	2 ms	Geophones	Trough
Halfmile	New Brunswick, Canada	2 ms	Geophones	Trough
Sudbury	Ontario, Canada	2 ms	Geophones	Trough

Two assets chosen for in-depth analysis

Lalor (primary training asset)

Highest data quality, cleanest picks, 1-ms sampling gives fine-grained target. Establishes the in-asset ceiling for the method.

Halfmile (cross-asset target)

Different sampling rate, different sensors, different picking convention. A realistic test of generalization — not just a validation split.

What the data looks like

A shot gather: traces from one source, sorted by receiver offset

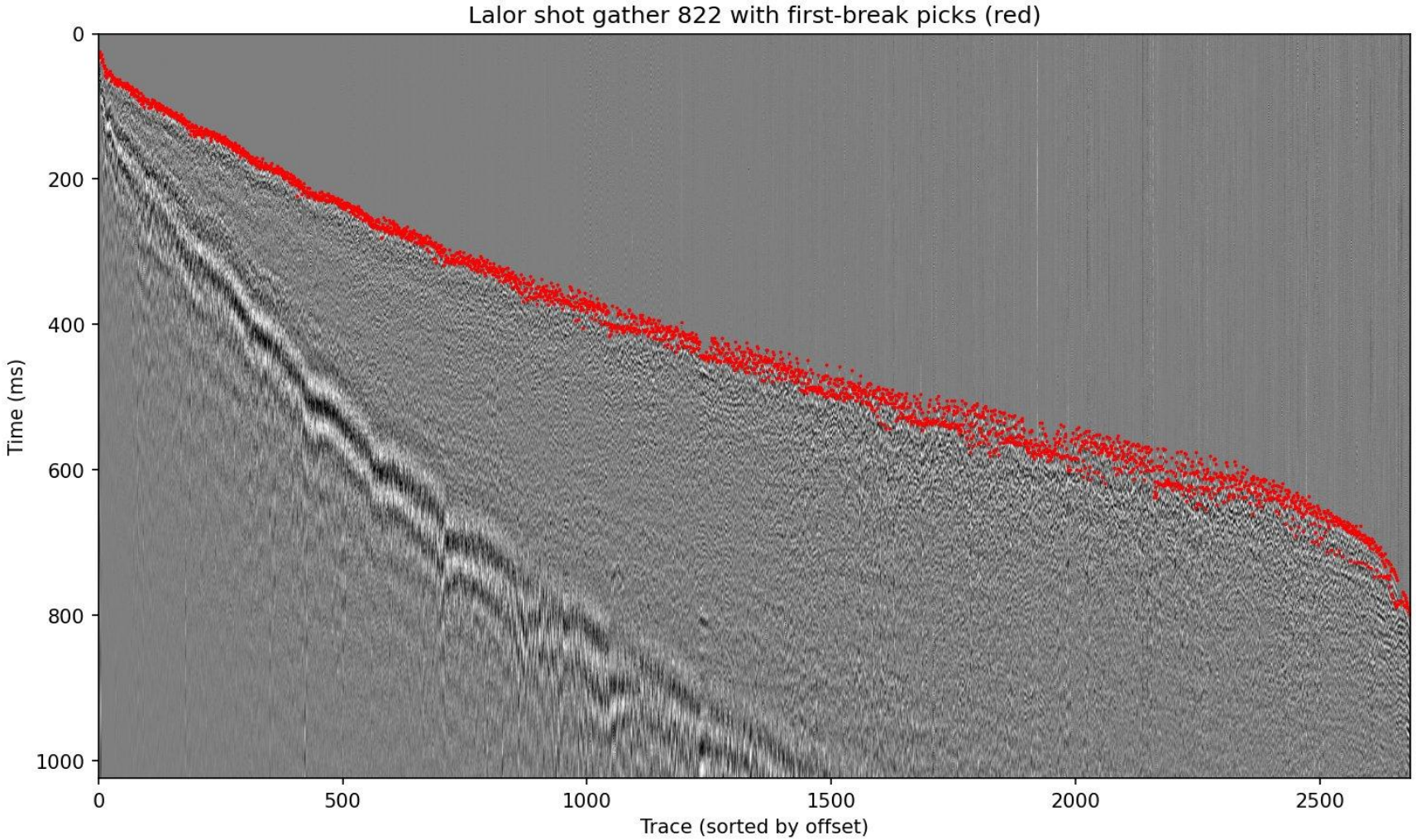
READING THIS IMAGE

X-axis:
Traces, sorted by total source-receiver offset

Y-axis:
Time, with zero at the shot moment

Grayscale:
Seismic amplitude (wiggles and reflections)

Red line:
Ground-truth first-break picks for every labeled trace



Lalor vs Halfmile: the numbers

Different scale, different sampling, different label density

Property	Lalor	Halfmile
Total traces	2,424,923	1,099,559
Samples per trace	1501	751
Sampling rate	1 ms	2 ms
Trace duration	1501 ms	1502 ms
Labeled fraction	50.0%	90.3%
Number of shots	907	690
Median traces / shot	2685	1604
First-break range	2–881 ms	11–1482 ms
Median first break	248 ms	344 ms

Key difference: Sampling rate. 1 ms vs 2 ms means the same model architecture can't see both assets the same way — we'll need to upsample Halfmile before inference.

Data pipeline

From flat HDF5 to ML-ready training patches

1. **HDF5 parse**
   Read trace data + metadata (shot ID, source XY, receiver XY, first-break time)

2. **Apply scale factors**
   SEG-Y conventions: COORD_SCALE, HT_SCALE applied before computing offsets

3. **Group by SHOTID**
   Reshape flat trace array into 2D shot gathers

4. **Sort by offset**
   Each shot's traces sorted by total source-receiver distance

5. **Split shot-level**
   85% train / 7.5% val / 7.5% test — split by SHOT, never by trace

6. **Extract patches**
   256-trace windows with stride 128; per-trace amplitude normalization

7. **Build masks**
   Binary target mask ±2 samples around each first-break pick

HDF5 structure

What each file contains and which fields matter

Key metadata fields (per trace):

SHOTID Shot identifier — used to group traces

SOURCE_X/Y Source coordinates (raw, pre-scale)

REC_X/Y Receiver coordinates (raw, pre-scale)

COORD_SCALE Scale factor: negative=divide, positive=multiply

SAMP_RATE Sampling rate in microseconds

SAMP_NUM Samples per trace

SPARE1 First-break time in ms (0 or -1 = unlabeled)

Validated assumption: file-level constants

Although stored per-trace, these fields should be constant within each file. I verified this explicitly before using them as scalars:

Field	Lalor	Halfmile
SAMP_RATE	1000 µs	2000 µs
SAMP_NUM	1501	751
COORD_SCALE	−10	+1
HT_SCALE	−10	−10000

Catching this variation is why the pipeline is asset-agnostic.

Patch extraction for training

Fixed-size windows ready for U-Net ingestion

256
traces per patch

1024
time samples per patch

128
stride (50% overlap)

±2
samples mask thickness

Design rationale

256 × 1024 patches:
Large enough for spatial context, small enough for 12 GB VRAM at batch 4.

50% overlap stride:
Doubles training data for free; first-break pixels appear in two patches.

Per-trace normalization:
Each trace divided by its 99th-percentile amplitude, clipped to [−5, 5].

Thick mask (±2 samples):
Gives the model a tolerance band; single-pixel targets are nearly unlearnable.

Resulting dataset sizes

Asset	Train	Val	Test
Lalor	8,999	751	775
Halfmile	6,389	559	581

Split by SHOT, not by trace — prevents leakage between train/test.

Classical baseline: STA/LTA

Before deep learning — what does a standard method achieve?

Short-Term-Average / Long-Term-Average

Idea:
Compute a rolling energy ratio between a short window (signal) and a long window (noise baseline). The first time the ratio exceeds a threshold marks the first arrival.

Parameters used:
`STA = 20 samples (20 ms)  |  LTA = 200 samples  |  threshold = 3.0`

Why use it?
Standard in seismology for decades. Deterministic, fast (~0.06 s per shot gather), interpretable. Sets a fair bar for any ML method to beat.

Known weaknesses:
No spatial context — each trace is picked independently. Pre-signal noise or coherent interference can trigger false picks. No notion of 'looks like a first break'.

Core computation

```
energy = trace²

STA[t] = mean(energy[t-20 : t])
LTA[t] = mean(energy[t-200 : t])

ratio[t] = STA[t] / LTA[t]

pick = first t where ratio[t] > 3.0
```

Vectorized over a batch of traces with cumulative-sum tricks for O(n) performance.

STA/LTA baseline results on Lalor

Evaluated on 50,305 labeled traces across 50 test shots

32.7 ms	110.0 ms	3.0 ms	62.4%
MAE	RMSE	Median |error|	Hit rate ≤ 5 ms

The gap between median and MAE is the story. Median |error| is 3 ms but MAE is 33 ms — the error distribution has a long heavy tail.

RMSE of 110 ms confirms it: STA/LTA is **precise when it works** but fails catastrophically on a minority of traces — typically noisy far-offset traces where pre-signal interference crosses the threshold before the real arrival.

Two ways to improve on this: (1) add spatial context so the model rejects outliers that disagree with neighbors, and (2) replace the threshold heuristic with a learned *what a first break looks like*.

Approach: U-Net segmentation

Cast first-break picking as a 2D segmentation problem

Reframing: Treat each shot gather as a 2D grayscale image. The first-break curve is a thin diagonal ridge running left-to-right. The model's job is to segment that ridge.

Per-trace pick extraction: argmax along the time axis for each column of the predicted probability map.

Why U-Net?

Standard for 2D semantic segmentation. Encoder-decoder with skip connections preserves fine spatial detail — critical when the target is a 1-pixel-thick line.

Why this formulation?

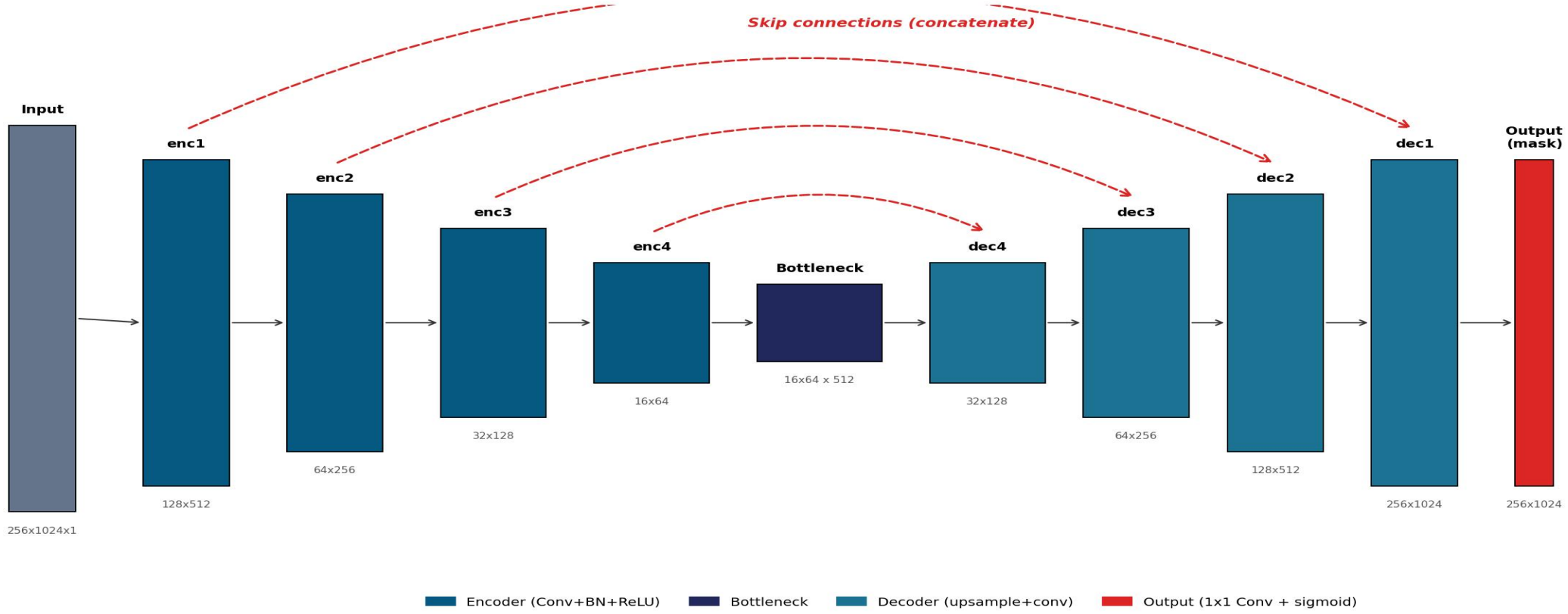
Spatial context built in — convolutional receptive field spans dozens of neighboring traces.
Outliers that disagree with neighbors are naturally suppressed.

Why 2D (not 1D)?

A per-trace 1D regressor discards the horizontal coherence. A 2D segmentation model can use the fact that adjacent traces have correlated first breaks.

U-Net architecture

4-level encoder / decoder, ~7.8 M parameters



Base filters	Levels	Parameters	Input	Output
32	**4 enc + 4 dec**	**7.77 M**	**256 x 1024 x 1**	**256 x 1024 x 1 (sigmoid)**

Loss function: weighted BCE + Dice

Handling severe class imbalance in the target mask

Class imbalance: First-break pixels are ~0.5% of the target mask. A model that predicts 'zero everywhere' achieves 99.5% pixel accuracy and is completely useless.

Weighted BCE

```
BCE = -(w·y·log p + (1-y)·log(1-p))
```

Positive class weight w = 50

Fights imbalance by making mistakes on first-break pixels 50x more costly than mistakes on background pixels.

Soft Dice Loss

```
Dice = 2·|y ∩ p| / (|y| + |p|)
Loss = 1 – Dice
```

Region-overlap metric

Provides a complementary gradient signal. Keeps pushing when BCE plateaus at low recall — the model is forced to actually produce ridges, not just minimize weighted pixel error.

Training methodology

25 epochs on 9K Lalor patches, RTX 4070

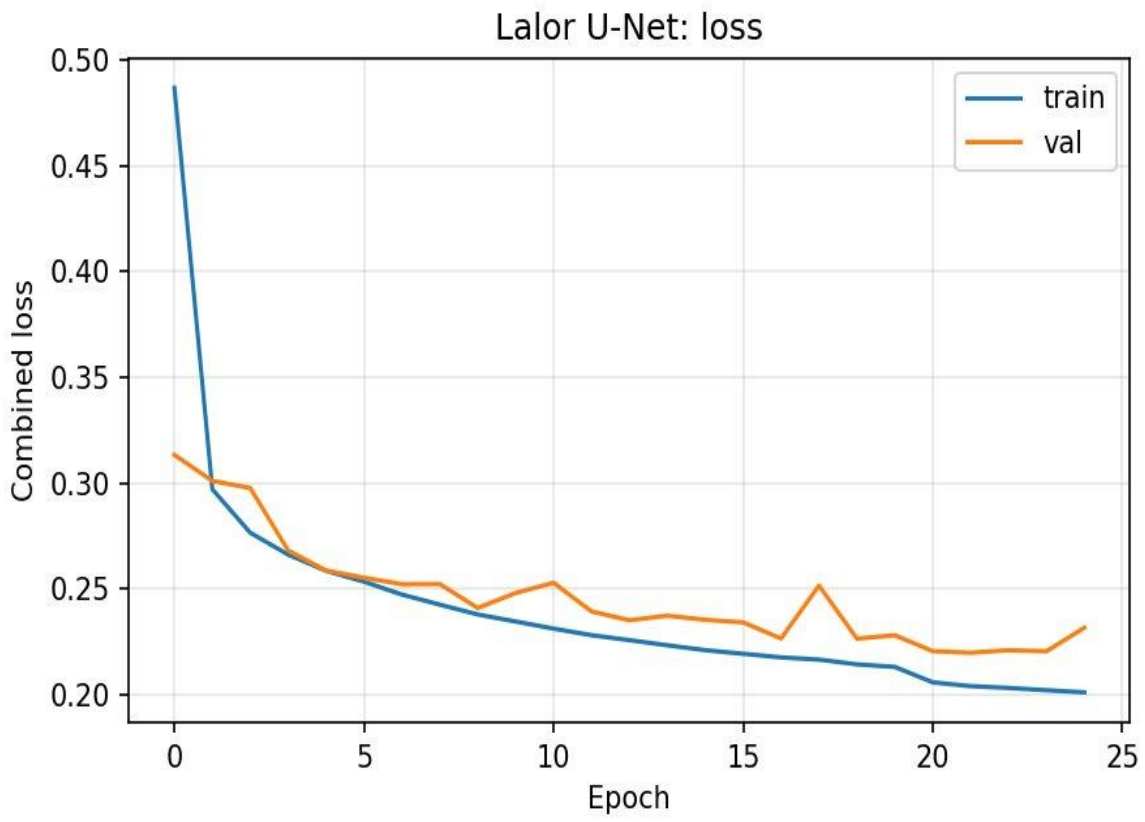
Optimizer	Adam, lr = 1e-3
Batch size	4
Epochs	25 (EarlyStopping p=7)
LR schedule	ReduceLROnPlateau (0.5×, p=3)
Augmentation	Horizontal flip, amplitude ±30%, polarity flip
Train time	~2 h 55 m

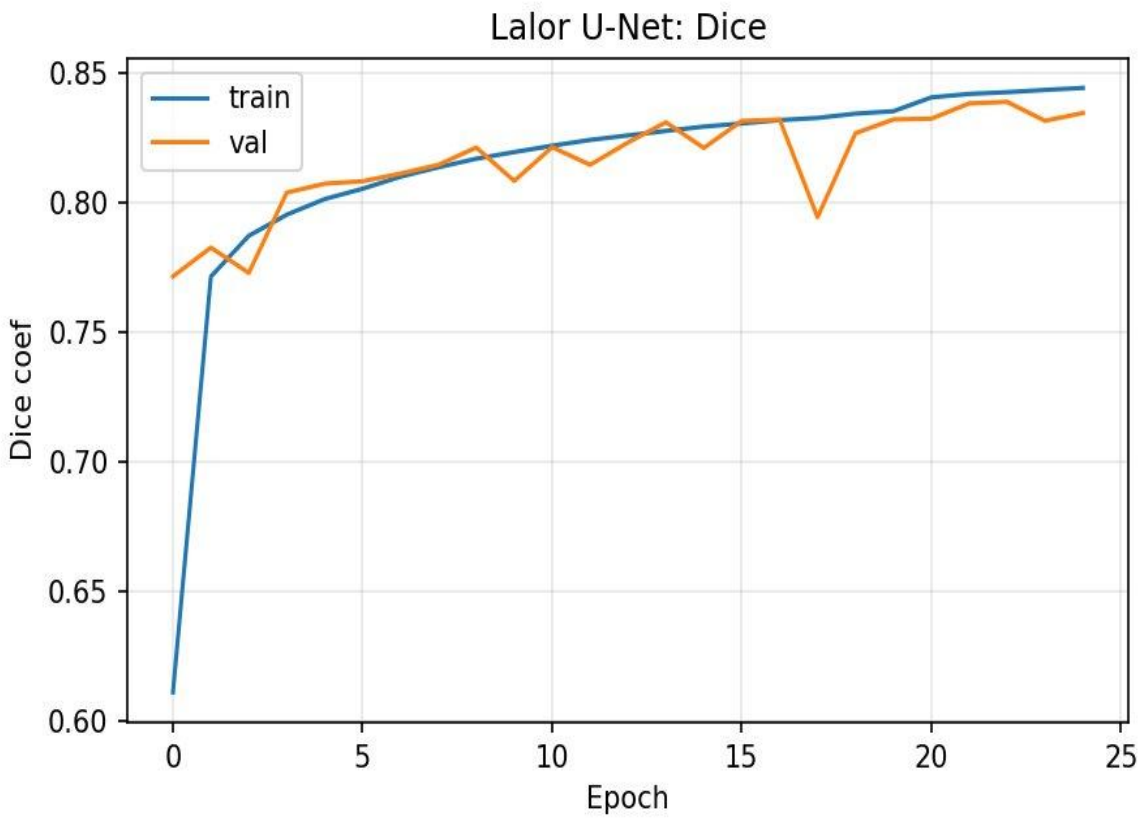
Augmentation rationale

Only geometrically-valid transforms: horizontal flip (V-symmetry of shot gathers), polarity flip (seismic is sign-invariant for FB), amplitude scaling (robustness to acquisition gain).

Training curves — Lalor U-Net

Clean convergence, small train/val gap indicates good generalization





Final val Dice: **0.838** · Final val loss: **0.220** · Train–val gap: **negligible (no overfitting)**

Lalor test-set results

Held-out 69 shots · 88,321 labeled traces · never seen by the model

1.13 ms	1.71 ms	96.1%	98.5%
MAE	RMSE	Hit rate ≤ 2 ms	Hit rate ≤ 5 ms

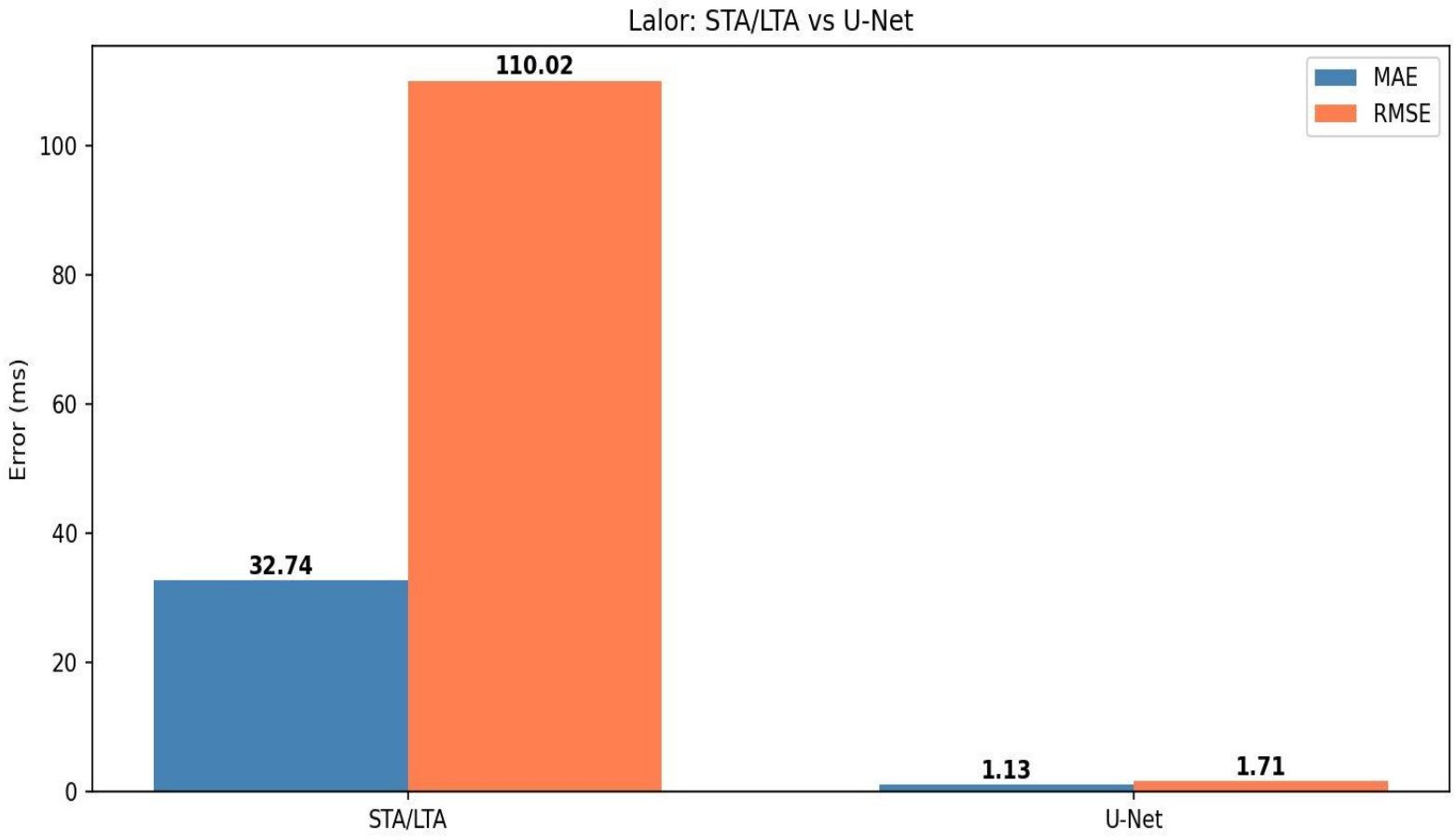
vs STA/LTA baseline: 29× better MAE · 64× better RMSE · +36 percentage points hit-rate

The U-Net's RMSE (1.71 ms) is only 1.5× its MAE (1.13 ms) — ***no long tail of catastrophic failures.***

STA/LTA's RMSE/MAE ratio was 3.4 — its errors were dominated by outliers.

STA/LTA vs U-Net — head to head

Same test set, same metric definitions



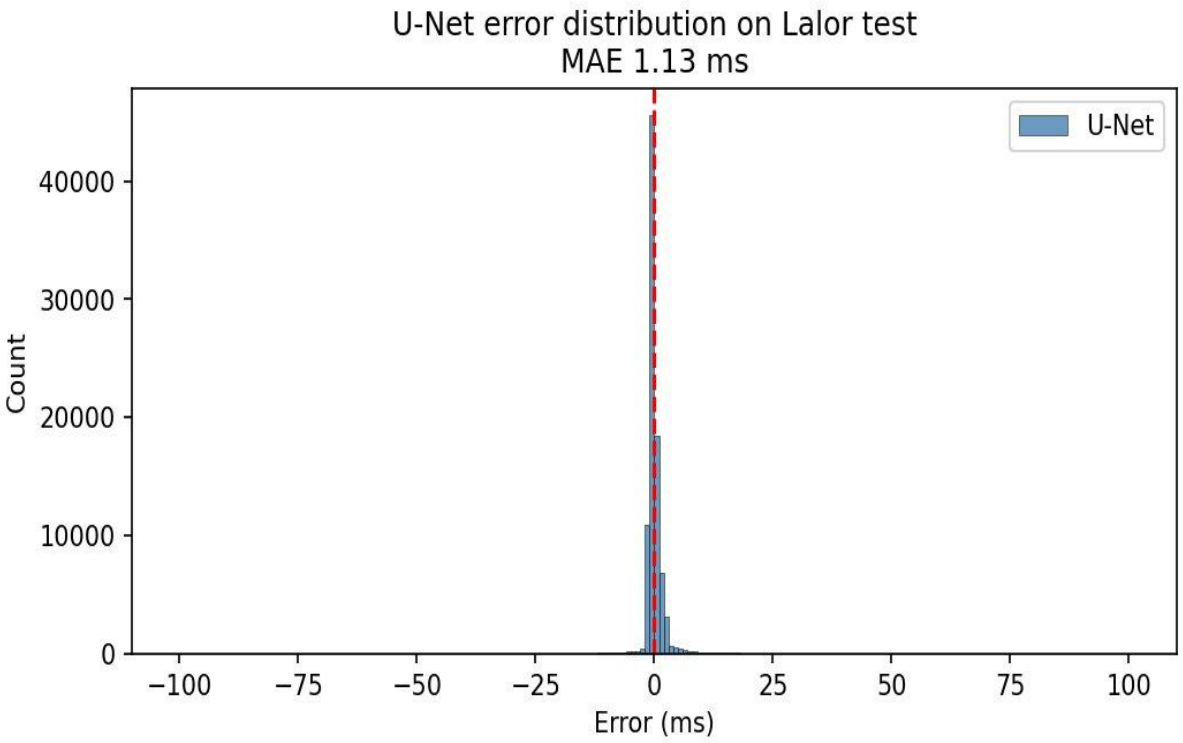
Bar heights are MAE (blue) and RMSE (coral) in ms.

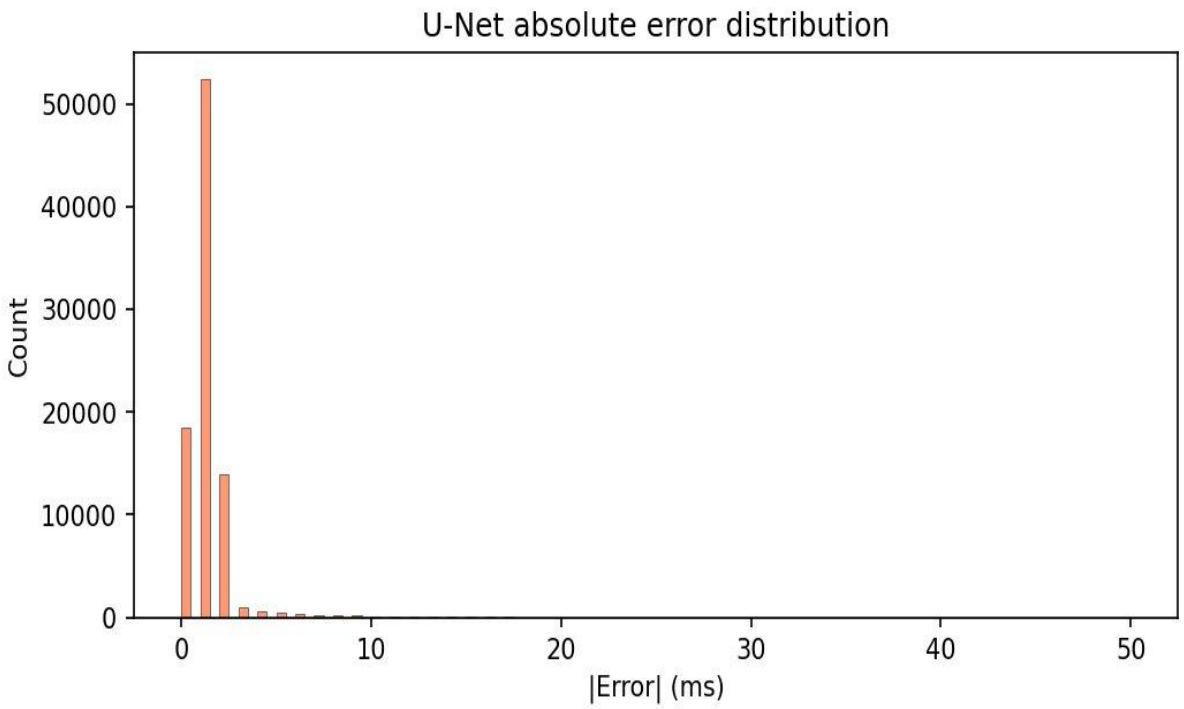
Improvement

MAE	**×29**
RMSE	**×64**
Hit ≤ 2 ms	**+52.5 pp**
Hit ≤ 5 ms	**+36.1 pp**
Hit ≤ 10 ms	**+27.9 pp**
Hit ≤ 20 ms	**+22.7 pp**

Error distribution on Lalor

Virtually no catastrophic failures





Reading the distributions:

Left: signed error clusters tightly at zero with essentially no tails. Bias = −0.53 ms (nearly zero).
Right: 98.5% of absolute errors are under 5 ms; the distribution falls off steeply after 2 ms.

The cross-asset question

Does the Lalor model work on data it's never seen?

> *...accurately detect first breaks across different seismic assets,* ***regardless of varying signal-to-noise ratios*** *and* ***variation of first-break line behavior*** *across the different seismic assets.*
>
> — Task description

Sampling rate

Lalor = 1 ms
Halfmile = 2 ms

Model input format is incompatible unless we resample.

Hardware / wavelet

Lalor: MEMS accelerometers
Halfmile: geophones

Different sensor response = different wavelet shape.

Picking convention

Lalor: onset annotation
Halfmile: trough annotation

Ground truth is defined differently between surveys.

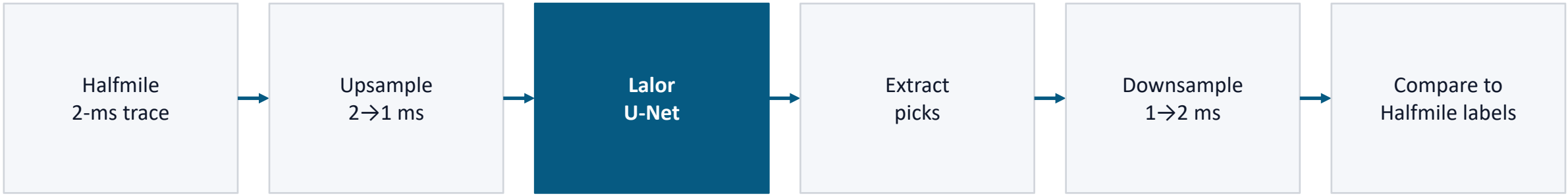
Experimental design

A 2×2 matrix that answers three different questions

	Evaluated on Lalor	Evaluated on Halfmile
Lalor-trained U-Net	**A — In-asset baseline** Establishes ceiling. Is the method any good?	**B — Zero-shot transfer** Do learned features transfer with no additional training?
Halfmile fine-tuned	**C — Forgetting check** Does fine-tuning on Halfmile destroy the original Lalor performance?	**D — Fine-tuning payoff** How much does one hour of fine-tuning help?

Zero-shot transfer: methodology

Run the Lalor model on Halfmile — with minimal adaptation



Implementation details

Polyphase resampling (scipy.signal.resample_poly) — properly anti-aliased, maintains spectral content.

Same normalization as Lalor training (per-trace 99th-percentile, clip to [−5, 5]).

No retraining, no weight updates — the point is to isolate how far the features transfer on their own.

Zero-shot results

Decent transfer with a clear, interpretable failure mode

13.72 ms	6.0 ms	33.1%	−6 ms
MAE on Halfmile test	Median |error|	Hit ≤ 5 ms	Systematic bias

Interpretation

Partial transfer confirmed:
Median error of 6 ms on zero-shot is respectable — U-Net features like 'a coherent wavefront arrives here' are asset-agnostic.

But there's a systematic bias of −6 ms:
Predictions are consistently early. This matches the Lalor-onset vs Halfmile-trough convention difference — roughly half a wavelength = ~6 ms.

The model hasn't failed — it's doing its job correctly using Lalor's definition of 'first break'. Fixing this requires showing it Halfmile's convention.

Fine-tuning methodology

Teach the Lalor model Halfmile's conventions, without losing its Lalor skills

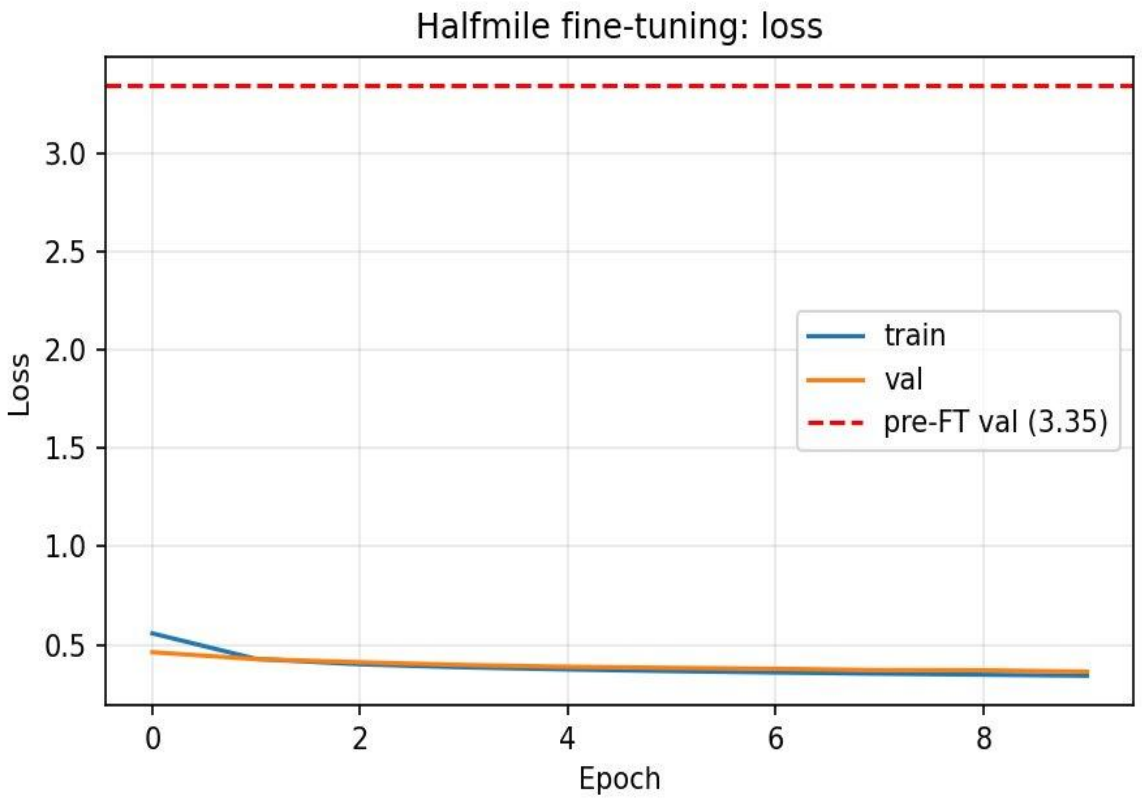
Starting weights	Lalor-trained U-Net (epoch 22)
Layers frozen	None — all 7.77 M parameters updated
Learning rate	1e-4 (10× lower than scratch)
Epochs	10 max, EarlyStopping patience 4
Training data	6,389 Halfmile patches (upsampled to 1 ms)
Train time	~50 min on RTX 4070

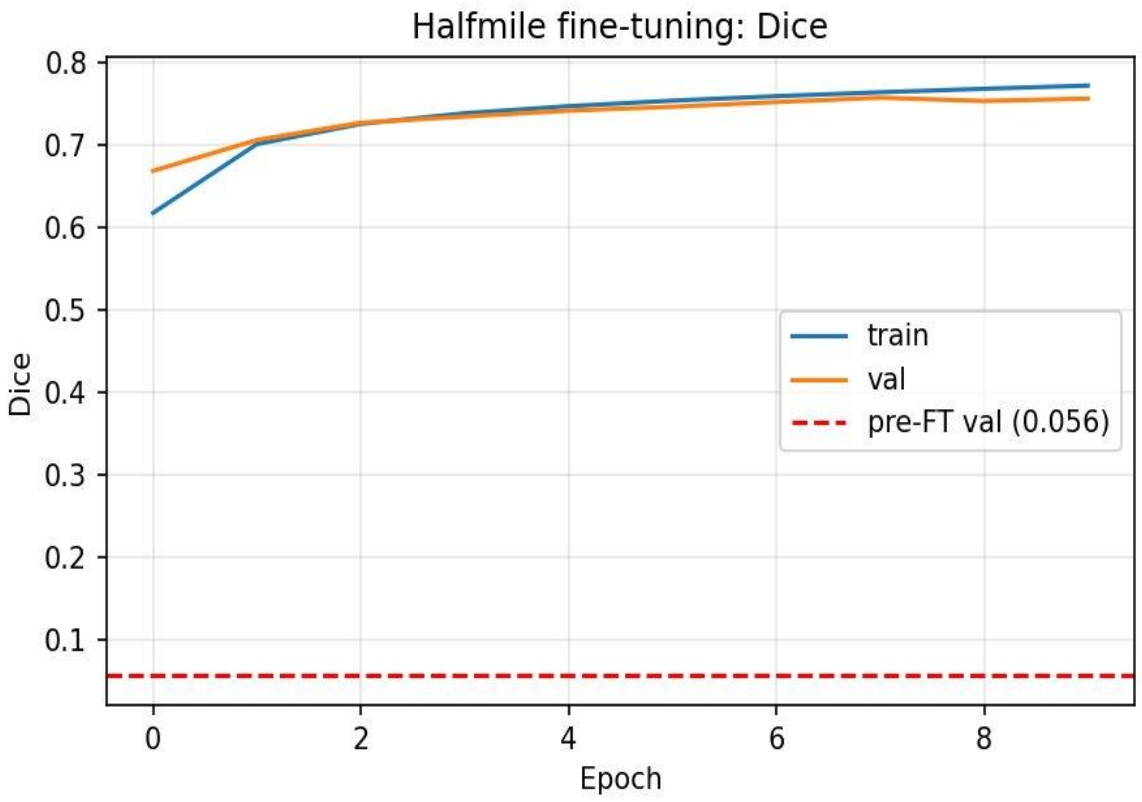
Why 10× lower learning rate?
A normal learning rate (1e-3) would overwrite the transferred features in a few steps. 1e-4 lets the model adapt without forgetting.

Why all layers unfrozen?
Freezing the encoder would preserve Lalor features but prevent adaptation to Halfmile wavelet shapes. The low LR already provides enough regularization.

Fine-tuning — rapid adaptation

Pre-FT baseline marked in red; both train and val improve immediately





Pre-FT val Dice: 0.056 → Post-FT val Dice: 0.756 · 13× improvement in 50 min

Cross-asset MAE matrix

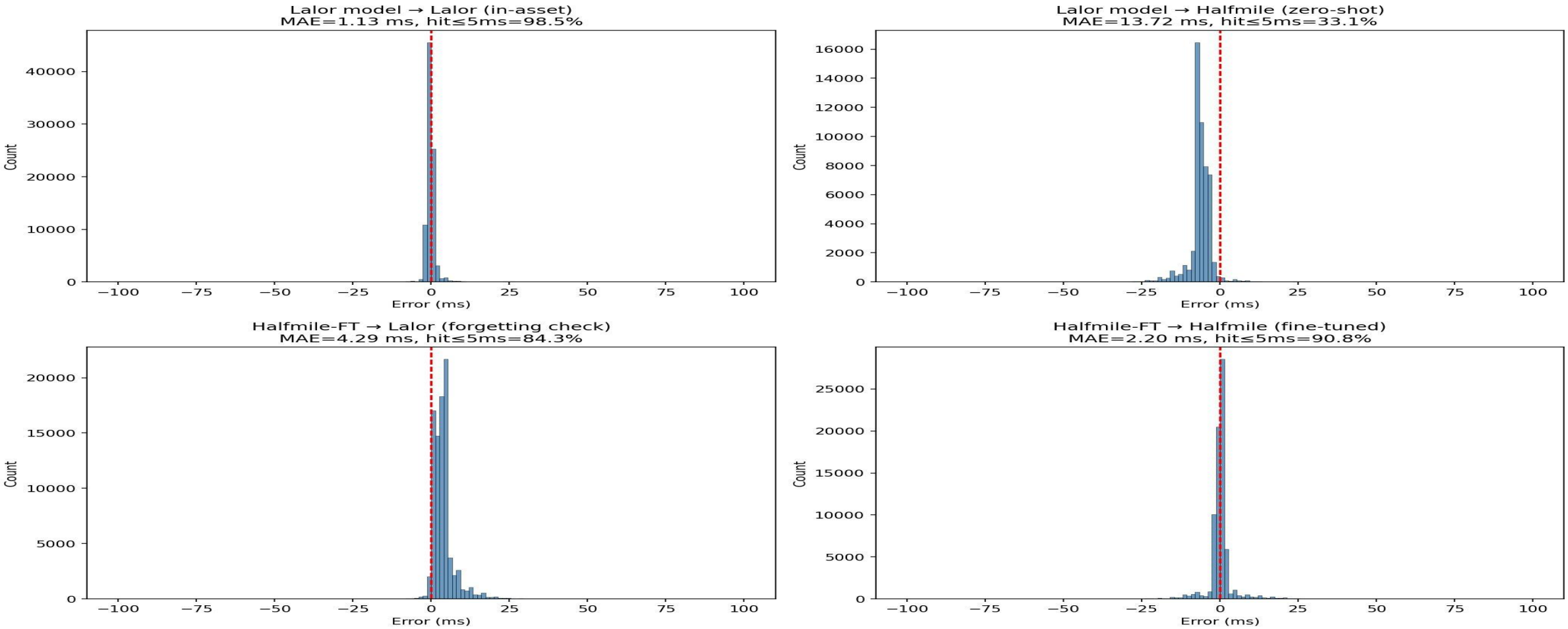
Four cells, four answers

	Lalor test	Halfmile test
Lalor model	**1.13** ms MAE *In-asset gold standard*	**13.72** ms MAE *Zero-shot — biased early*
Halfmile-FT model	**4.29** ms MAE *No catastrophic forgetting*	**2.20** ms MAE *Fine-tuned payoff*

The Halfmile-FT model is effectively a two-asset model — 4.29 ms on Lalor is worse than the dedicated model's 1.13 but still acceptable for production use.

Error distributions — all four combinations

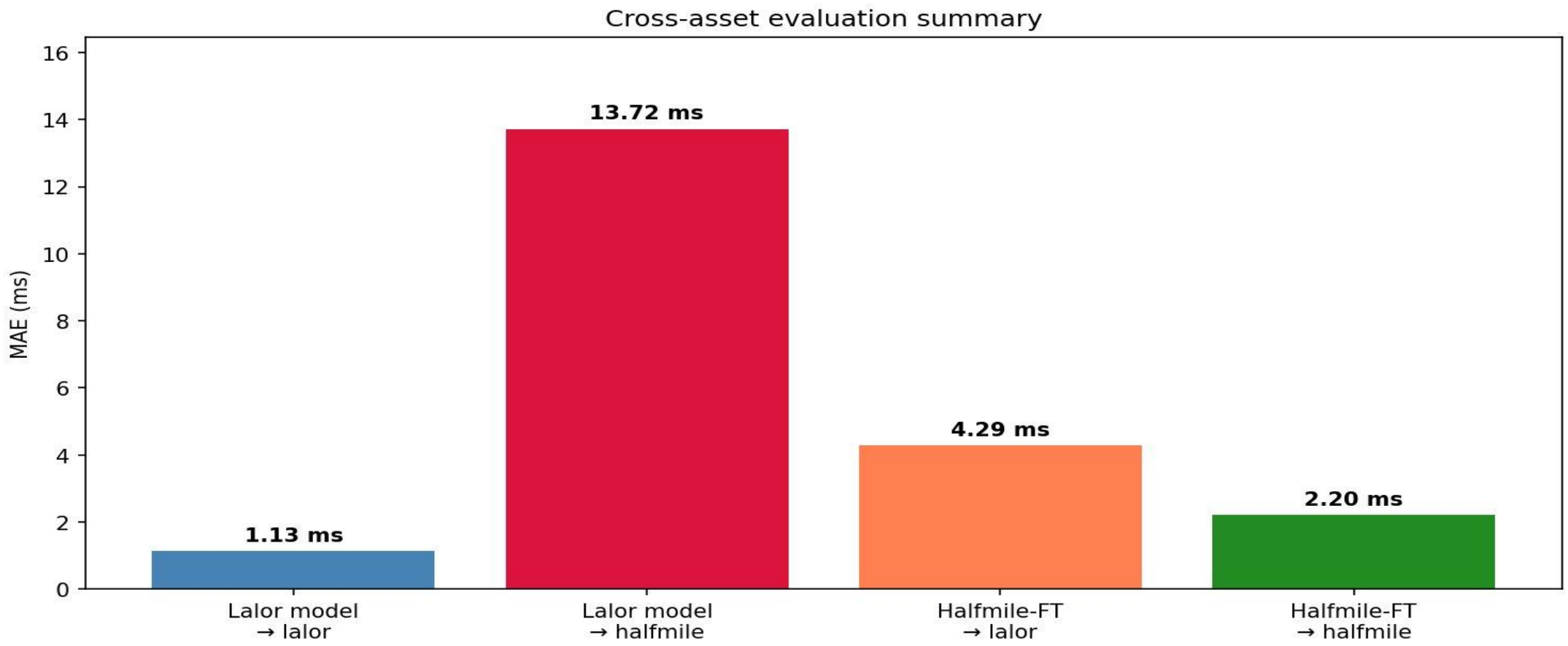
The systematic bias in zero-shot transfer is visible and interpretable



Top-right panel shows the zero-shot failure mode clearly: errors are skewed toward negative values (predictions early).

Summary at a glance

Fine-tuning is worth the 50-minute compute cost



Key take-away: Fine-tuning beats zero-shot 6× on the target asset and only modestly degrades the source asset.

Limitations

Honest assessment of what this work does and doesn't do

Only two of four assets

Brunswick and Sudbury weren't evaluated. The pipeline is asset-agnostic so extending is straightforward, but was out of time budget.

Sampling mismatch via resampling

Upsampling Halfmile to 1 ms is a pragmatic choice. Training a native 2-ms variant or using multi-scale inputs might do better.

Picking conventions learned implicitly

The onset-vs-trough offset is learned by fine-tuning, not handled as a principled preprocessing step.

No uncertainty estimation

Production use wants per-pick confidence so a geophysicist can triage edge cases. MC-dropout or calibrated thresholds would be straightforward.

Future work

Directions to improve generalization and robustness

- **Joint multi-asset training**
  Train a single model on Lalor + Halfmile + Brunswick + Sudbury simultaneously, rather than sequentially fine-tuning.

- **Domain-adversarial training**
  Add a domain classifier on U-Net features, adversarially trained to be uninformative. Forces asset-invariant representations.

- **Uncertainty estimation**
  MC-dropout or deep ensembles to produce per-pick confidence intervals, enabling triage of low-confidence predictions.

- **Physics-informed features**
  Feed the model with source-receiver offset as an extra channel. First-break time correlates strongly with offset; explicit conditioning should help.

- **Self-supervised pretraining**
  Pretrain on unlabeled shot gathers via masked-patch reconstruction before supervised fine-tuning — lets us leverage the 50% of unlabeled traces.

Summary

1.13 ms	29×	2.20 ms	4.29 ms
Lalor in-asset MAE	vs STA/LTA baseline	Halfmile fine-tuned	Halfmile-FT on Lalor (no forgetting)

*A production-grade pipeline that **achieves sub-sample accuracy in-asset** and **generalizes with minimal fine-tuning**.*

Thank you

Rufat Mahmudov · April 2026